



Trabajo Práctico N°2

Generador PRBS, Filtro FIR, Unidad de Diagnóstico (DU),
Medidor de Clock y Mapa de Registros

Curso de Diseño Digital VLSI - Fundación Fulgor

Damián Lugano

1 de septiembre de 2026

Índice

1. Introducción	1
1.1. Objetivos	1
1.2. Arquitectura general	1
2. Bloque A: Generador PRBS (prbs_parallel)	2
2.1. Descripción y especificaciones	2
2.2. Paralelización del generador	2
2.3. Implementación RTL	4
2.4. Verificación	5
3. Bloque B: Filtro FIR (fir_parallel)	5
3.1. Descripción general	5
3.2. Paralelización del filtro	5
3.3. Implementación RTL	6
3.4. Verificación	6
4. Bloque C: Unidad de diagnóstico (du)	8
4.1. Objetivo y especificaciones	8
4.2. Arquitectura e implementación RTL	8
4.3. Verificación	10
5. Bloque D: Medidor de frecuencia (freq_meas)	10
5.1. Objetivo	10
5.2. Principio de funcionamiento	10
5.3. Arquitectura e implementación RTL	11
5.4. Cruces de dominio de clock	12
5.5. Medición de un clock de 8 GHz	12
5.6. Verificación	12
6. Bloque E: Mapa de registros (regmap)	13
6.1. Objetivo	13
6.2. Arquitectura e interfaz	13
6.3. Transacciones de acceso	13
6.4. Mapa de direcciones	14
6.5. Verificación	15
7. Integración del sistema (top)	15
7.1. Arquitectura e integración	15
7.2. Verificación	16
7.3. Resultados de síntesis	17
8. Conclusiones	19

1. Introducción

1.1. Objetivos

El objetivo de este trabajo es el diseño de un sistema capaz de generar un estímulo, procesarlo, y luego observarlo con una ventana de captura, todo controlado a través de un mapa de registros, siendo este la única conexión entre el software y el hardware. Además, este modulo integrará un modulo de medición de frecuencia, para chequear que la frecuencia de ambos clock del sistema sea la esperada.

1.2. Arquitectura general

El diseño contemplará 5 bloques principales, cada uno con una función definida. Además, contará con 2 dominios de reloj: CPU y DSP. Cada modulo trabajará en un dominio de clock, o incluso en los dos, por lo que este diseño integrará también, multiples técnicas de CDC (Clock Domain Crossing).

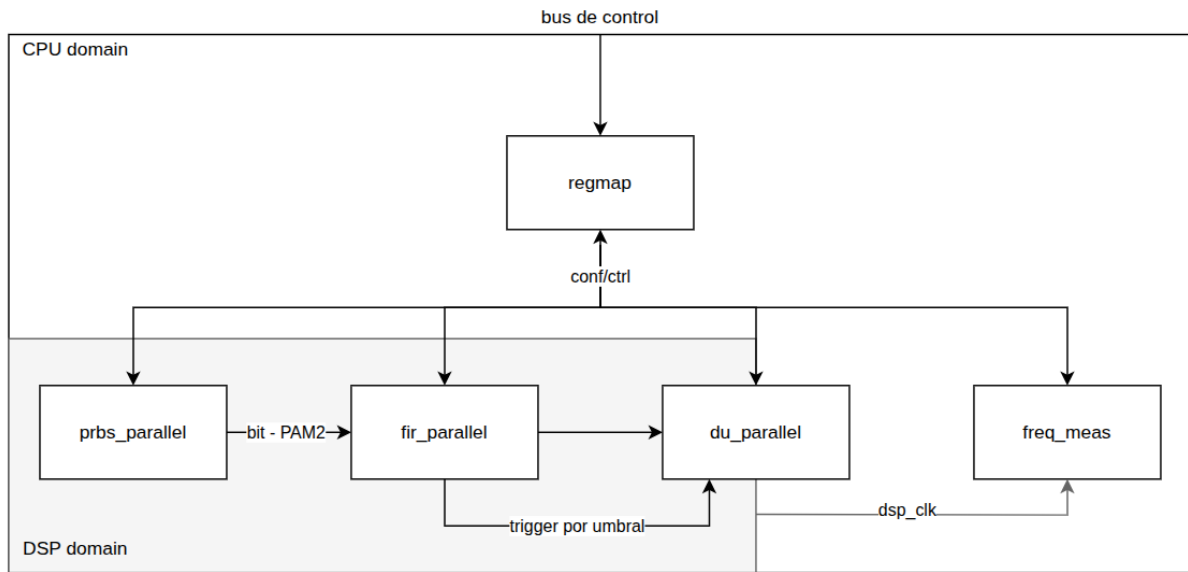


Figura 1: Esquema simple de la arquitectura general del sistema

Bloque A: Generador PRBS

El generador PRBS es un generador de secuencias binarias pseudo-aleatorias basado en un LFSR (Linear Feedback Shift Register). El mismo tendrá implementados 6 polinomios de PRBS de 10 a 15, que podrán ser seleccionados mediante configuración desde el regmap.

En este caso, se hace una implementación paralela del generador PRBS, de 4 salidas. La salida de este módulo será modulada a PAM-2 a través de un submodulo y será el input del filtro FIR paralelo.

Su funcionamiento será principalmente en el clock DSP.

Bloque B: Filtro FIR

Se utilizará el mismo filtro FIR del TP N°1, un filtro RRC(Root Raised Cosine) de 19 taps, aunque en esta oportunidad se le hicieron modificaciones a su arquitectura para permitir una paralelización de 4.

Como entrada toma la secuencia pseudoaleatoria del PRBS modulada a PAM-2 y su salida irá a la DU para su visualización. Tanto sus coeficientes, como su señal de enable se configurarán mediante el regmap.

Su funcionamiento será principalmente en el clock DSP.

Bloque C: DU (Diagnostic Unit)

Debe pensarse a la DU como un osciloscopio digital, dentro del mismo chip. Su funcionamiento se basa en capturar una señal a su entrada luego de su disparo, que será en este caso por umbral.

Una vez disparado, en función de su modo de captura, tomará una cierta cantidad de muestras hasta completar su memoria. Luego de eso, su memoria podrá ser leída secuencialmente mediante el regmap.

En este caso, al tratarse de una implementación paralela, se instanciarán 4 DUs, que se dispararán con la misma señal de trigger en simultaneo y capturarán la señal de forma paralela.

Su funcionamiento será principalmente en el clock DSP, aunque la lectura de la memoria será sincrónica al clock de CPU.

Bloque D: Medidor de frecuencia de clock

El medidor de frecuencia de clock es un modulo que, si bien no forma parte del datapath, es fundamental para la verificación de un clock de dominio desconocido, en base a uno conocido.

Brevemente, su funcionamiento se basa en dos contadores simultaneos, uno por cada dominio de clock, que contarán durante una ventana de tiempo de referencia generada a partir del clock conocido. Una vez terminada, se podrá calcular la relación entre cuentas del clock desconocido, y la ventana de tiempo configurada.

En este caso, el clock conocido o de referencia será el clock de CPU, mientras que el clock a medir será el de DSP.

Bloque E: Mapa de registros

El mapa de registros es la interfaz de entrada/salida entre el SW y el sistema en HW. Mediante el regmap, se podrá configurar, controlar y leer los resultados de todos los modulos dentro del chip.

Su funcionamiento será principalmente en el clock de CPU, ya que es el nexo entre este y el chip.

2. Bloque A: Generador PRBS (prbs_parallel)

2.1. Descripción y especificaciones

El módulo implementado genera secuencias binarias pseudo-aleatorias mediante un registro de desplazamiento con realimentación lineal (LFSR). El orden de la secuencia es configurable entre PRBS10 y PRBS15 y el estado inicial se determina mediante una semilla no nula, configurable.

Con el objetivo de paralelizar el sistema, el generador produce cuatro bits consecutivos de la secuencia PRBS en cada ciclo del clock DSP. Por lo tanto, el diseño posee un grado de paralelismo $P = 4$. La selección del polinomio y la semilla se realizan desde el mapa de registros y son capturadas al inicio una nueva secuencia.

Polinomios implementados

La Tabla 1 presenta los seis polinomios implementados y el valor utilizado para seleccionarlos. Cada término del polinomio determina uno de los taps que intervienen en la realimentación del LFSR. Las sumas se realizan en el campo de GF(2), por lo que se implementan mediante operaciones XOR.

Cuadro 1: Polinomios implementados por el generador PRBS.

i_sel	Patrón	Polinomio
000	PRBS10	$x^{10} + x^7 + 1$
001	PRBS11	$x^{11} + x^9 + 1$
010	PRBS12	$x^{12} + x^6 + x^4 + x + 1$
011	PRBS13	$x^{13} + x^4 + x^3 + x + 1$
100	PRBS14	$x^{14} + x^5 + x^3 + x + 1$
101	PRBS15	$x^{15} + x^{14} + 1$

2.2. Paralelización del generador

Representación matricial

El comportamiento de un LFSR puede expresarse como un sistema lineal sobre GF(2). Si $\mathbf{s}[k]$ representa su estado en el instante k y A es la matriz de transición correspondiente al polinomio seleccionado, el avance de un generador serial queda definido por

$$\mathbf{s}[k+1] = A \mathbf{s}[k]. \quad (1)$$

Para obtener P bits por ciclo no es necesario instanciar cuatro veces el generador PRBS. En cambio, se eleva la matriz de transición a la potencia correspondiente al grado de paralelismo:

$$\mathbf{s}[k+P] = A^P \mathbf{s}[k]. \quad (2)$$

En este diseño $P = 4$, por lo que el próximo estado se calcula mediante A^4 . Un script desarrollado en Python construye la matriz A a partir de los taps, se calcula su cuarta potencia en GF(2) y permite obtener las ecuaciones XOR utilizadas en el RTL.

Ejemplo para PRBS10

Para mostrar el procedimiento se considera el polinomio de PRBS10

$$p(x) = x^{10} + x^7 + 1. \quad (3)$$

La matriz de transición es:

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}. \quad (4)$$

Al calcular la cuarta potencia de esta matriz en GF(2) se obtiene:

$$A^4 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}. \quad (5)$$

Las filas de A^4 determinan las ecuaciones del próximo estado. Expresadas con la numeración utilizada en el RTL resultan:

$$\begin{aligned} q_9[k+4] &= q_5[k], & q_4[k+4] &= q_0[k], \\ q_8[k+4] &= q_4[k], & q_3[k+4] &= q_9[k] \oplus q_6[k], \\ q_7[k+4] &= q_3[k], & q_2[k+4] &= q_8[k] \oplus q_5[k], \\ q_6[k+4] &= q_2[k], & q_1[k+4] &= q_7[k] \oplus q_4[k], \\ q_5[k+4] &= q_1[k], & q_0[k+4] &= q_6[k] \oplus q_3[k]. \end{aligned} \quad (6)$$

Los cuatro bits entregados durante el ciclo son $q_9[k]$, $q_8[k]$, $q_7[k]$ y $q_6[k]$. Estos corresponden a cuatro valores consecutivos del generador PRBS. Luego de presentarlos en la salida, el registro avanza directamente al estado $\mathbf{s}[k+4]$. La Figura 2 muestra el circuito resultante para este polinomio.

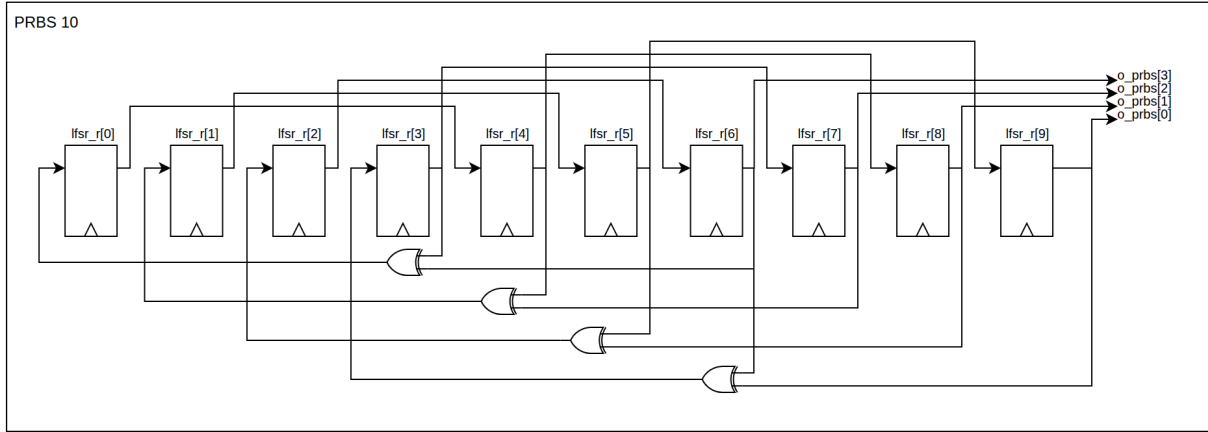


Figura 2: Implementación paralela de grado cuatro para PRBS10.

2.3. Implementación RTL

El datapath utiliza un banco de registros de 15 bits, correspondiente al mayor orden soportado. Para cada polinomio seleccionado, el bloque combinacional calcula el próximo estado mediante las ecuaciones obtenidas de la matriz A^4 . Los bits que no pertenecen al orden seleccionado se mantienen fuera del estado activo. Por otro lado, otro bloque combinacional selecciona los cuatro bits más significativos del LFSR correspondiente y los presenta en las salidas paralelas.

La selección activa se almacena en `sel_r`, el estado del LFSR en `lfsr_r` y la condición de ejecución en `running_r`. La semilla y el polinomio se capturan únicamente ante un pulso de inicio. Esto evita que una escritura de configuración realizada mientras el generador está activo modifique la secuencia en curso. La arquitectura RTL sintetizada se muestra en la Figura 3.

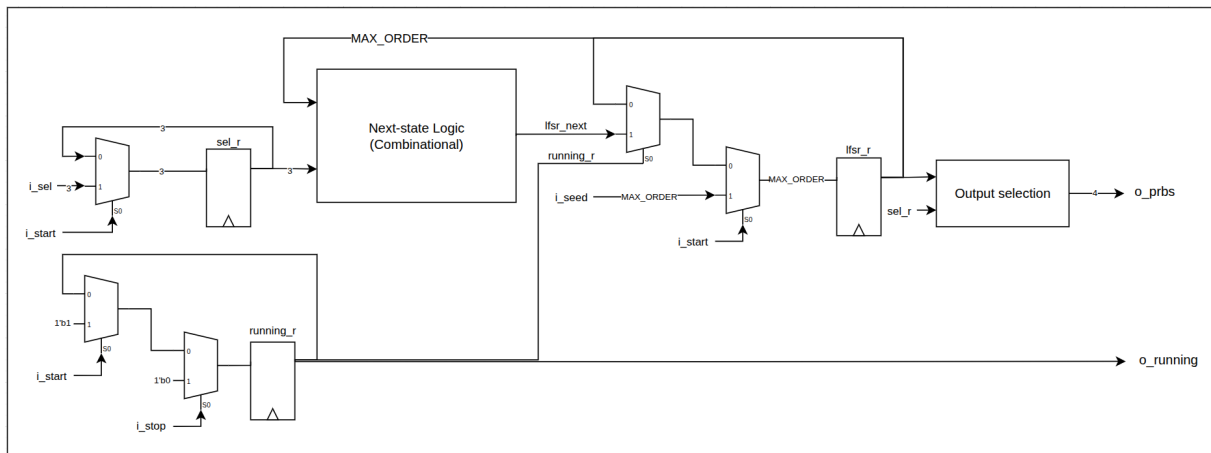


Figura 3: Arquitectura RTL del generador PRBS configurable.

Secuencia de configuración

La configuración del generador debe realizarse antes de iniciar una secuencia. En primer lugar se escribe una semilla no nula en `PRBS_SEED`. Luego se escribe en `PRBS_CTRL[4:2]` la selección indicada en la Tabla 1 y se genera un pulso de START mediante el bit 0 del mismo registro. Al recibir este pulso en el dominio DSP, el módulo captura la semilla y la selección, carga el estado inicial y activa `RUNNING`.

Mientras el generador se encuentra activo entrega cuatro bits y avanza cuatro posiciones de la secuencia por cada ciclo. Para detenerlo se genera un pulso de STOP mediante `PRBS_CTRL[1]`. El estado queda congelado hasta recibir un nuevo START, momento en el que se vuelven a cargar la configuración y la semilla. El estado de ejecución puede consultarse en `PRBS_STATUS[0]`.

2.4. Verificación

La verificación se realizó en dos partes. Inicialmente se realizó un testbench básico para recorrer un período completo de cada uno de los polinomios implementados. Para cada orden se usó una semilla con todos sus bits en uno. Las salidas del RTL se compararon bit a bit con secuencias generadas mediante un modelo de referencia desarrollado en Python.

Después, se implementó un testbench más robusto basado en clases tipo UVM. Las clases desarrolladas fueron: un generador que crea configuraciones aleatorias válidas de polinomio, semilla y duración; un driver que aplica la secuencia de reset, configuración, START y STOP; y un monitor que registra los cuatro bits producidos en cada ciclo. Finalmente, las muestras capturadas se comparan con el modelo de referencia de Python.

De esta manera se verificó tanto la correspondencia de las secuencias paralelas con el modelo serial como el comportamiento de control del módulo frente a configuraciones aleatorias.

3. Bloque B: Filtro FIR (fir_parallel)

3.1. Descripción general

Para el módulo de filtro FIR se utilizó el diseño del filtro RRC (Root Raised Cosine) de 19 taps desarrollado en el TP1. Se conservó principalmente la arquitectura de la suma de productos del diseño original, que aprovechaba simetría y entrada PAM-2 utilizando arquitectura folded y muxes como multiplicadores.

Sin embargo, en este caso por el paralelismo implementado en el datapath, la entrada está compuesta por cuatro símbolos PAM-2 consecutivos y la salida contiene las cuatro muestras filtradas correspondientes.

La Figura 4 presenta un FIR genérico de ocho taps. Este ejemplo se utiliza únicamente para explicar el proceso de paralelización; la misma idea se generaliza luego al filtro de 19 taps empleado en este caso.

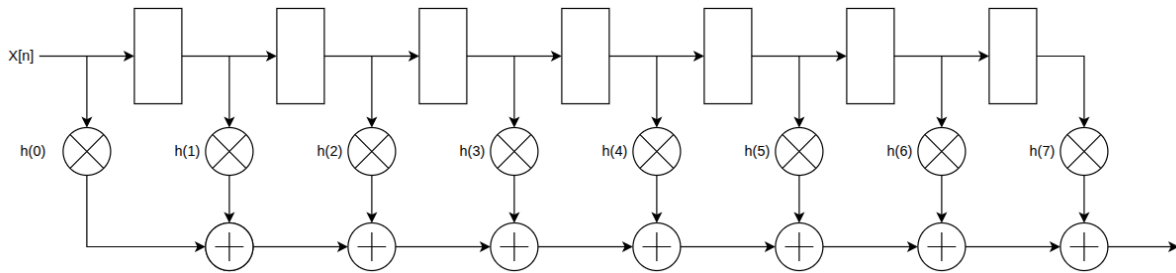


Figura 4: Estructura de un filtro FIR genérico de ocho taps.

3.2. Paralelización del filtro

La salida de un filtro FIR de N taps se expresa como:

$$y[n] = \sum_{k=0}^{N-1} h_k x[n-k]. \quad (7)$$

Para obtener cuatro resultados por ciclo se escribieron explícitamente las ecuaciones temporales del filtro, agrupando las entradas y salidas de a cuatro. A continuación se muestra el procedimiento para el ejemplo de ocho taps de la Figura 4. En el primer ciclo ingresan x_0 , x_1 , x_2 y x_3 , y las salidas correspondientes son:

$$\begin{aligned} y_0 &= x_0 h_0 + x_{-1} h_1 + x_{-2} h_2 + x_{-3} h_3 + x_{-4} h_4 + x_{-5} h_5 + x_{-6} h_6 + x_{-7} h_7 \\ y_1 &= x_1 h_0 + x_0 h_1 + x_{-1} h_2 + x_{-2} h_3 + x_{-3} h_4 + x_{-4} h_5 + x_{-5} h_6 + x_{-6} h_7 \\ y_2 &= x_2 h_0 + x_1 h_1 + x_0 h_2 + x_{-1} h_3 + x_{-2} h_4 + x_{-3} h_5 + x_{-4} h_6 + x_{-5} h_7 \\ y_3 &= x_3 h_0 + x_2 h_1 + x_1 h_2 + x_0 h_3 + x_{-1} h_4 + x_{-2} h_5 + x_{-3} h_6 + x_{-4} h_7 \end{aligned}$$

Durante el segundo ciclo ingresan x_4, x_5, x_6 y x_7 . Las cuatro nuevas salidas combinan esas muestras con las entradas almacenadas del ciclo anterior:

$$\begin{aligned} y_4 &= x_4h_0 + x_3h_1 + x_2h_2 + x_1h_3 + x_0h_4 + x_{-1}h_5 + x_{-2}h_6 + x_{-3}h_7 \\ y_5 &= x_5h_0 + x_4h_1 + x_3h_2 + x_2h_3 + x_1h_4 + x_0h_5 + x_{-1}h_6 + x_{-2}h_7 \\ y_6 &= x_6h_0 + x_5h_1 + x_4h_2 + x_3h_3 + x_2h_4 + x_1h_5 + x_0h_6 + x_{-1}h_7 \\ y_7 &= x_7h_0 + x_6h_1 + x_5h_2 + x_4h_3 + x_3h_4 + x_2h_5 + x_1h_6 + x_0h_7 \end{aligned}$$

En el tercer ciclo, las primeras cuatro entradas ya se desplazaron por la memoria del filtro. Las ecuaciones resultan:

$$\begin{aligned} y_8 &= x_8h_0 + x_7h_1 + x_6h_2 + x_5h_3 + x_4h_4 + x_3h_5 + x_2h_6 + x_1h_7 \\ y_9 &= x_9h_0 + x_8h_1 + x_7h_2 + x_6h_3 + x_5h_4 + x_4h_5 + x_3h_6 + x_2h_7 \\ y_{10} &= x_{10}h_0 + x_9h_1 + x_8h_2 + x_7h_3 + x_6h_4 + x_5h_5 + x_4h_6 + x_3h_7 \\ y_{11} &= x_{11}h_0 + x_{10}h_1 + x_9h_2 + x_8h_3 + x_7h_4 + x_6h_5 + x_5h_6 + x_4h_7 \end{aligned}$$

Al comparar las ecuaciones puede observarse que cada salida necesita una ventana de ocho muestras consecutivas, pero el punto inicial de esa ventana cambia para cada lane. Las muestras más recientes provienen directamente de las cuatro entradas del ciclo actual y las restantes deben recuperarse de la memoria del filtro. Una vez identificado este patrón, la construcción puede extenderse a cualquier cantidad de taps y a sucesivos grupos de cuatro entradas.

La Figura 5 muestra el resultado de aplicar este razonamiento al FIR genérico de ocho taps. Se obtienen cuatro caminos de suma de productos, uno para cada salida, alimentados por las entradas actuales y por las muestras conservadas en el regresor.

3.3. Implementación RTL

El submódulo **regressor** conserva la historia necesaria para construir las cuatro ventanas de convolución. En lugar de utilizar una única cadena que se desplaza una muestra por ciclo, se implementan cuatro cadenas de registros paralelas, una por lane de entrada. Cada nuevo ciclo desplaza simultáneamente cuatro muestras dentro de la memoria.

Para un filtro de N taps se requieren $N - 1$ muestras anteriores. El número de niveles del regresor se calcula como

$$L = \left\lceil \frac{N - 1}{P} \right\rceil. \quad (8)$$

En el diseño utilizado, $N = 19$ y $P = 4$, por lo que deben almacenarse 18 muestras en cinco niveles de registros. Los primeros cuatro registros reciben las entradas del ciclo en orden temporal inverso, de la más reciente a la más antigua. En cada nivel siguiente se guarda el grupo proveniente del nivel anterior.

El módulo **fir_parallel** combina esta memoria con las entradas actuales para formar el array de muestras que será el input del modulo **sop** para cada lane. De esta manera se obtienen cuatro resultados consecutivos por ciclo.

3.4. Verificación

La verificación del filtro FIR paralelo se realizó utilizando un testbench básico. En el mismo se le ingresa una señal PAM-2 aleatoria y se comprueba su salida haciendo vector matching con una señal obtenida del golden reference diseñado en python.

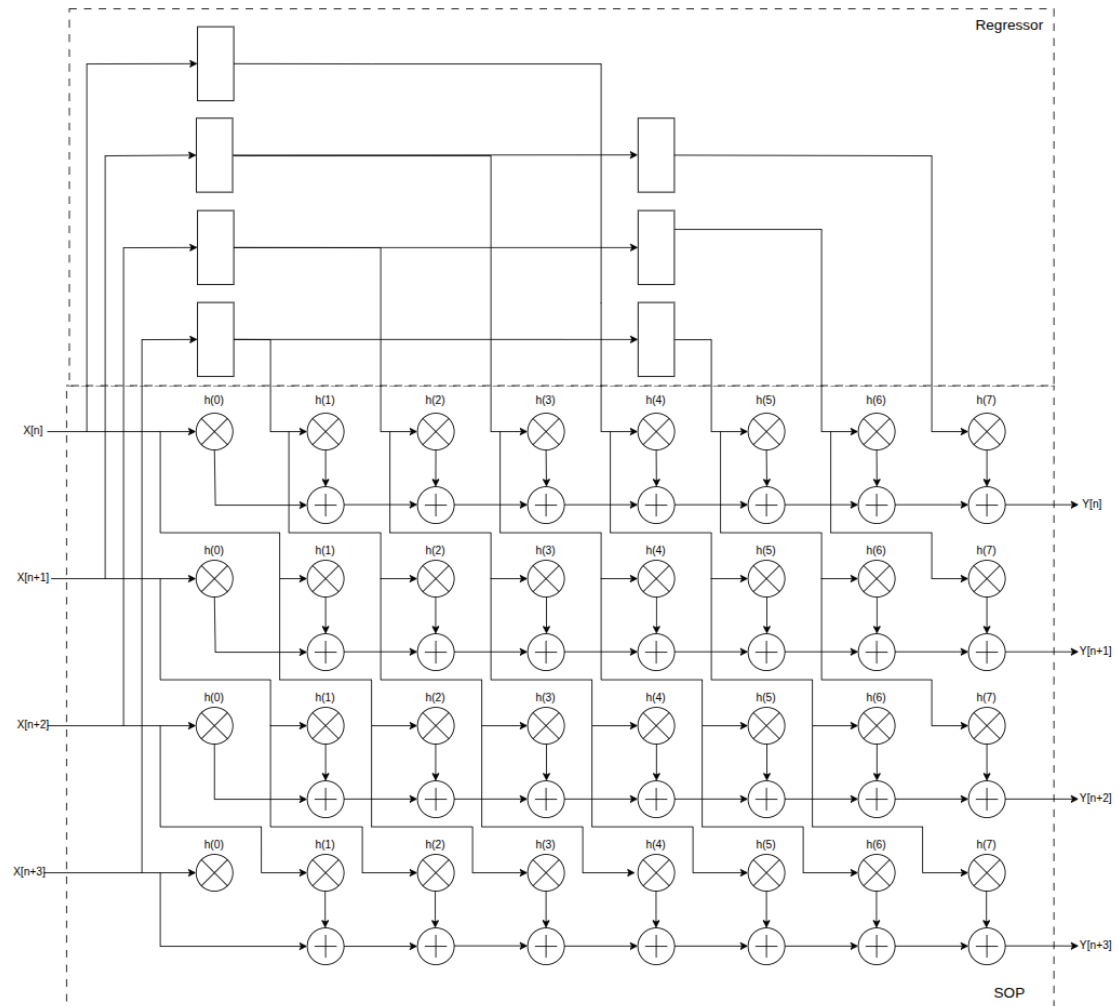


Figura 5: Paralelización por cuatro del FIR genérico de ocho taps.

4. Bloque C: Unidad de diagnóstico (du)

4.1. Objetivo y especificaciones

La unidad de diagnóstico (DU) funciona como un osciloscopio digital integrado en el sistema. Su objetivo es almacenar una ventana temporal de la señal de salida del filtro FIR alrededor de un evento de disparo. En el top se instancian cuatro DUs, una por cada lane del datapath, todas controladas por la misma señal de ARM y por un trigger común.

Cada instancia posee una memoria de 256 palabras de 12 bits. El trigger se genera cuando al menos una de las cuatro salidas del FIR supera el umbral configurado, quedando la posición del evento de disparo dentro de la ventana depende del modo seleccionado.

Los modos de captura de la DU se resumen en la Tabla 2.

Cuadro 2: Modos de captura de la DU. ($W = 256$)

Valor	Modo	Comportamiento
00	PRE	Captura W muestras continuamente y finaliza al recibir el trigger
01	MID	Conserva $W/2$ muestras anteriores y $W/2$ posteriores al trigger
10	POST	Comienza la captura de W muestras al recibir el trigger.

En modo PRE la memoria funciona como un buffer circular desde el ARM, capturando continuamente hasta el evento de trigger. En modo MID también se escribe como un buffer circular antes del trigger, pero luego del trigger la captura continúa hasta completar media memoria más. Para una profundidad de 256 posiciones, la ventana queda formada por 128 muestras anteriores y 128 posteriores al trigger. En modo POST no se escribe antes del evento sino que la muestra que produce el trigger es la primera de la ventana y luego se completa el resto de la memoria.

El modo se captura junto con el pulso de ARM. Si se recibe el valor reservado 11, el módulo selecciona MID como modo seguro por defecto.

4.2. Arquitectura e implementación RTL

La arquitectura de la DU se basa en una máquina de estados que controla la adquisición de muestras y su escritura en memoria, en función del modo de captura. Esto naturalmente se realiza en el dominio de DSP, mientras que la lectura de la memoria se realiza en el dominio de CPU una vez la captura finalizó.

Máquina de estados

La máquina de estados posee los estados IDLE, RUN, CAPTURING y DONE. Su funcionamiento se muestra en la Figura 6.

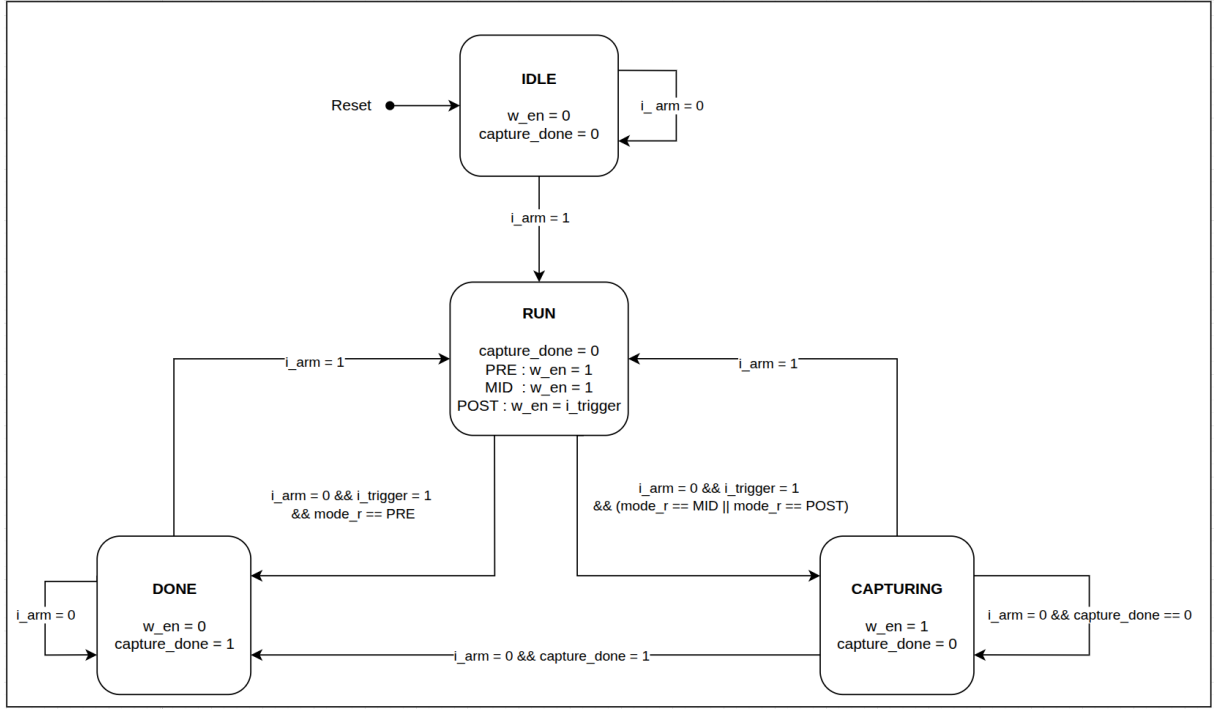


Figura 6: Máquina de estados de la DU.

Las ecuaciones de control de la máquina de estados se expresan a continuación:

$$w_{en} = \begin{cases} \text{state_r} == \text{RUN}, & \text{modo PRE,} \\ (\text{state_r} == \text{RUN}) \parallel (\text{state_r} == \text{CAPTURING}), & \text{modo MID,} \\ ((\text{state_r} == \text{RUN}) \ \&\& \ i_trigger) \parallel (\text{state_r} == \text{CAPTURING}), & \text{modo POST} \end{cases} \quad (9)$$

El valor objetivo del contador posterior al trigger se determina mediante:

$$\text{target} = (\text{mode_r} == \text{MID}) ? ((\text{DEPTH}/2) - 1) : (\text{DEPTH} - 1) \quad (10)$$

Por lo tanto, la condición que finaliza la etapa de captura es:

$$\text{capture_done} = (\text{state_r} == \text{CAPTURING}) \ \&\& \ (\text{post_tr_cnt_r} == \text{target}), \quad (11)$$

$$\text{done_wr} = (\text{state_r} == \text{DONE}) \quad (12)$$

En PRE no se utiliza el contador posterior al trigger porque la máquina pasa directamente de RUN a DONE.

Escritura y lectura de memoria

La organización de la memoria se muestra en la Figura 7. La escritura es síncrona al clock DSP y se realiza cuando $w_{en} = 1$. Durante un nuevo ARM la escritura se frena para que el puntero y la FSM regresen a su estado inicial.

$$\text{mem}[\text{ptr_wr}] \leq i_data \quad \text{si} \quad w_{en} \ \&\& \ !i_arm \quad (13)$$

El puntero **ptr_wr** se incrementa con cada escritura, utilizando un contador que hace overflow al llegar al tamaño de la memoria. Cuando la memoria está completa, el puntero indica la dirección de la muestra más antigua, permitiendo reconstruir la captura en orden cronológico.

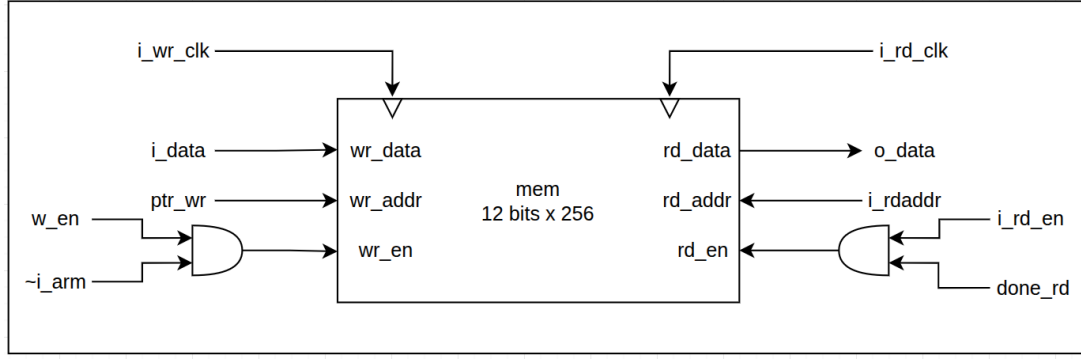


Figura 7: Organización de los puertos de escritura y lectura de la memoria de la DU.

La señal `done_wr` se sincroniza hacia el dominio CPU, con un sincronizador de nivel en base a FF. La lectura solo se habilita cuando esta señal sincronizada está activa, evitando accesos mientras la memoria aún está siendo modificada. El proceso de lectura es síncrono, ante `i_rd_en` y una dirección válida, el dato se registra en `o_data` y `o_rd_valid` se activa durante la respuesta. El puntero final también se mantiene estable y se entrega al mapa de registros para que el software determine la primera dirección que debe leer.

4.3. Verificación

La DU se verificó mediante un testbench con clocks independientes para los dominios DSP y CPU. Se aplicó como entrada una secuencia PRBS conocida y se ejecutó una adquisición completa en cada uno de los modos PRE, MID y POST. Para cada caso se armó la DU, se generó el trigger, se esperó al flag de done y se leyó la memoria completa desde la posición informada por el puntero. Las 256 muestras recuperadas se compararon con las posiciones esperadas de la secuencia de referencia, comprobando tanto la ubicación temporal del trigger como el orden de lectura y el funcionamiento del buffer circular.

5. Bloque D: Medidor de frecuencia (freq_meas)

5.1. Objetivo

El medidor de frecuencia permite determinar la frecuencia de un clock desconocido a partir de otro clock cuya frecuencia es conocida. En este sistema se utiliza el clock de CPU de 200 MHz como referencia para medir el clock DSP, cuya frecuencia es 1 GHz. Aunque este bloque no forma parte del datapath, permite comprobar desde software que ambos clocks están funcionando correctamente.

5.2. Principio de funcionamiento

El principio de medición consiste en generar, dentro del dominio de referencia, una ventana con una duración conocida. Mientras la ventana está activa se cuentan los flancos ascendentes del clock a medir. Si la ventana permanece activa durante W ciclos del clock de referencia, su duración es:

$$T_W = \frac{W}{f_{ref}} \quad (14)$$

Si durante ese intervalo se cuentan N_{in} ciclos del clock desconocido, su frecuencia puede estimarse como:

$$\hat{f}_{in} = \frac{N_{in}}{T_W} = \frac{N_{in} f_{ref}}{W} \quad (15)$$

Para $f_{ref} = 200$ MHz y una frecuencia de entrada nominal de $f_{in} = 1$ GHz, la cantidad esperada de ciclos dentro de la ventana es

$$N_{in} = W \frac{f_{in}}{f_{ref}} = 5W \quad (16)$$

Como la medición se realiza contando flancos, la menor variación observable en el resultado corresponde a una cuenta. Esto produce una variación mínima en la frecuencia estimada de $\Delta f = f_{ref}/W$. La resolución relativa se obtiene normalizando este incremento respecto de la frecuencia de entrada:

$$\frac{\Delta f}{f_{in}} = \frac{f_{ref}}{W, f_{in}} = \frac{1}{N_{in}} \quad (17)$$

Para obtener un error de 1ppm (10^{-6}) se requiere entonces:

$$\frac{1}{5W} < 10^{-6} \implies W > 200000 \quad (18)$$

Por lo que se eligió $W = 200\,000$ como valor de ventana. A 200 MHz esto corresponde a una ventana de 1 ms, dentro de la cual se esperan:

$$N_{in} = 5 \cdot 200\,000 = 1\,000\,000 \quad (19)$$

ciclos del clock DSP.

El ancho mínimo de cada contador se obtiene calculando la cantidad de bits necesaria para representar su máximo valor:

$$B_W = \lceil \log_2(W + 1) \rceil = \lceil \log_2(200\,001) \rceil = 18, \quad (20)$$

$$B_C = \lceil \log_2(N_{in} + 1) \rceil = \lceil \log_2(1\,000\,001) \rceil = 20. \quad (21)$$

El RTL utiliza $WIN_W=19$, incorporando un bit de margen para la longitud programable de la ventana, y $CNT_W=20$ para el contador del clock de entrada. Con una cuenta nominal de un millón, este último permanece por debajo de su máximo valor representable, $2^{20} - 1 = 1\,048\,575$.

5.3. Arquitectura e implementación RTL

La arquitectura del frecuencímetro se muestra en la Figura 8. El dominio de referencia contiene el registro de configuración de la ventana, el contador de ciclos de CPU y la lógica que genera el nivel `window_open_ref_r`. Ante un pulso de `i_start`, el módulo captura `i_window_len`, reinicia el contador y abre una nueva ventana. El nivel permanece activo hasta que transcurre la cantidad de ciclos programada.

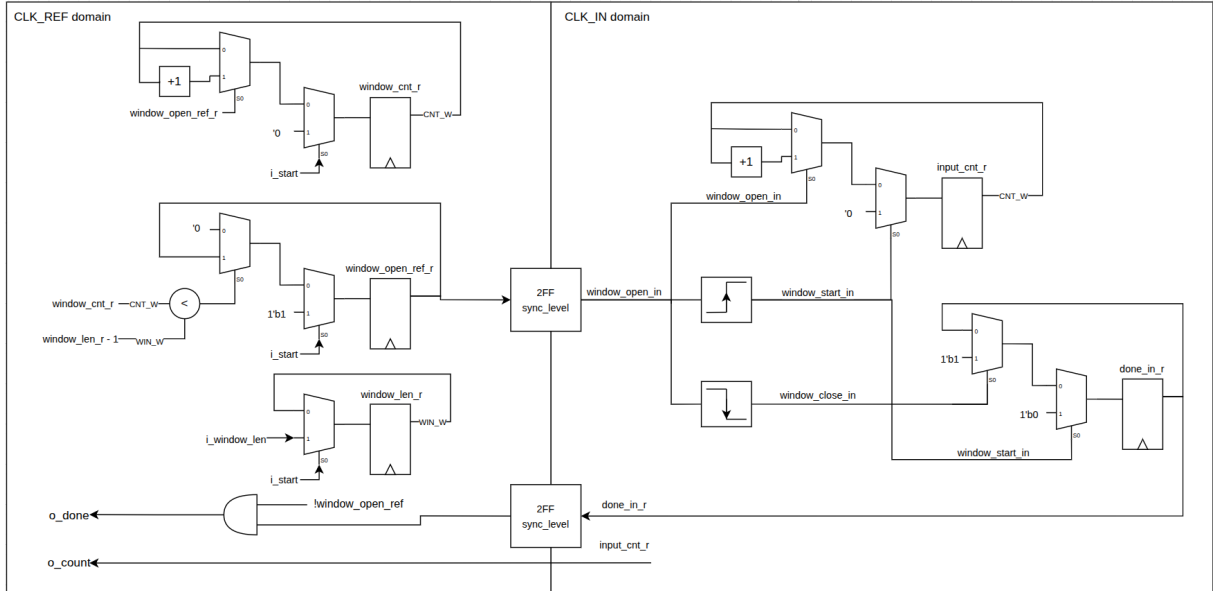


Figura 8: Arquitectura RTL del medidor de frecuencia.

La ventana se sincroniza hacia el dominio del clock de entrada. Allí, un detector de flanco ascendente genera el pulso que limpia el resultado anterior y un detector de flanco descendente identifica el final de la medición. Mientras el nivel sincronizado permanece activo, `input_cnt_r` se incrementa con cada flanco de `i_clk_in`. Al cerrarse la ventana, el contador queda congelado y se activa la indicación `done_in_r`.

Un nuevo pulso de `i_start` invalida inmediatamente el resultado previo. Además, la longitud de ventana se captura al comenzar la medición, por lo que una modificación posterior de `i_window_len` no afecta la cuenta en curso.

5.4. Cruces de dominio de clock

El diseño tiene dos cruces de dominio implementados mediante sincronizadores de FF. El primero transfiere el nivel `window_open_ref_r` desde el dominio de referencia hacia el dominio de entrada. Como la ventana permanece activa durante una gran cantidad de ciclos, el nivel puede atravesar el sincronizador sin problema. Los detectores de flanco generan luego pulsos de un ciclo ya síncronos a `i_clk_in`.

El segundo cruce transfiere el nivel `done_in_r` en sentido contrario. Esta señal se mantiene en uno desde que finaliza una medición hasta que comienza la siguiente, por lo que también puede sincronizarse como nivel sin problema.

El contador `input_cnt_r` es un bus de múltiples bits y se optó por no sincronizarlo bit a bit, sino que se utilizó una transferencia de tipo `bundled-data`. Cuando `done_in_r` se activa, el contador deja de modificarse y permanece estable hasta el próximo inicio. De esta manera, `o_done` funciona como flag de validez de `o_count`. La latencia del sincronizador da varios ciclos para que el dato se estabilice antes de ser observado desde el dominio CPU.

5.5. Medición de un clock de 8 GHz

Para conservar una resolución relativa de 1 ppm es necesario observar, como mínimo un millón de ciclos del clock a medir. A partir de la Ecuación 16, la longitud de ventana requerida para una frecuencia cualquiera puede calcularse como

$$W_{1\text{ ppm}} = \left\lceil 10^6 \frac{f_{ref}}{f_{in}} \right\rceil. \quad (22)$$

Para el caso de 8GHz planteado a en la consigna, utilizando un clock de 200 MHz se requieren

$$W_{8\text{ GHz}} = 10^6 \frac{200\text{ MHz}}{8\text{ GHz}} = 25\,000, \quad (23)$$

equivalente a 125 μs . Por lo tanto, desde el punto de vista de los anchos de los contadores, el clock de 8 GHz puede medirse sin modificaciones: la ventana de 25 000 ciclos entra en `WIN_W=19` y el resultado continúa siendo de aproximadamente un millón de cuentas, por debajo del máximo de `CNT_W=20`.

Sin embargo, es posible que la limitación sea de timing, ya que el contador debe poder operar físicamente a 8 GHz. Si el circuito no cierra timing a esa frecuencia, el método es matemáticamente válido pero la implementación no.

Son las frecuencias bajas las que exigen ventanas más largas para acumular el millón de cuentas necesario. Con `WIN_W=19`, la máxima ventana programable es de $2^{19} - 1 = 524\,287$ ciclos, o aproximadamente 2.62 ms a 200 MHz. Por lo tanto, la menor frecuencia para la cual puede alcanzarse una resolución de 1 ppm con los anchos actuales es aproximadamente

$$f_{in,\text{mín}} = \frac{10^6 f_{ref}}{2^{19} - 1} \approx 381.5\text{ MHz}. \quad (24)$$

Es decir, pueden medirse clocks más lentos, pero con una resolución relativa peor que 1 ppm.

5.6. Verificación

La verificación se realizó con clocks de 200 MHz y 1 GHz y una ventana de 200 000 ciclos de referencia. Se comprobó que la cuenta obtenida estuviera en el rango cercano de 1 000 000 ciclos y que `o_count` permaneciera estable mientras `o_done` estuviera activo. También se inició una segunda medición para verificar que `i_start` invalidara el resultado anterior y que un cambio de `i_window_len` durante la medición no modificara la ventana ya capturada.

El resultado obtenido fue el error de una cuenta, es decir 999 999.

6. Bloque E: Mapa de registros (regmap)

6.1. Objetivo

El mapa de registros es la interfaz entre el software y los distintos bloques de hardware del sistema. Su función es centralizar la configuración y el control de los módulos, permitiendo también la lectura de sus resultados y señales de estado. Para ello, decodifica las direcciones recibidas desde la CPU y las traduce en registros de configuración, pulsos de control o accesos a información generada por el hardware.

6.2. Arquitectura e interfaz

El bloque funciona de manera síncrona al clock de CPU y presenta una interfaz simple formada por el address del registro `i_addr`, dato de escritura `i_wdata`, request `i_req` y tipo de operación `i_is_write`. Como respuesta, entrega el dato de lectura `o_rdata` y la confirmación `o_ack`.

Internamente, la dirección selecciona el registro involucrado en la transacción. Los campos de configuración se almacenan en registros en el propio regmap, mientras que los campos de estado y resultados se obtienen directamente de los módulos correspondientes.

La Figura 9 muestra un esquema simplificado de la arquitectura. El dato de escritura se dirige al banco de registros seleccionado por `i_addr`; mientras que para las lecturas, un demultiplexor selecciona el valor que será registrado en `o_rdata`. La señal `o_ack` indica que la transacción fue atendida, en ambos casos por igual.

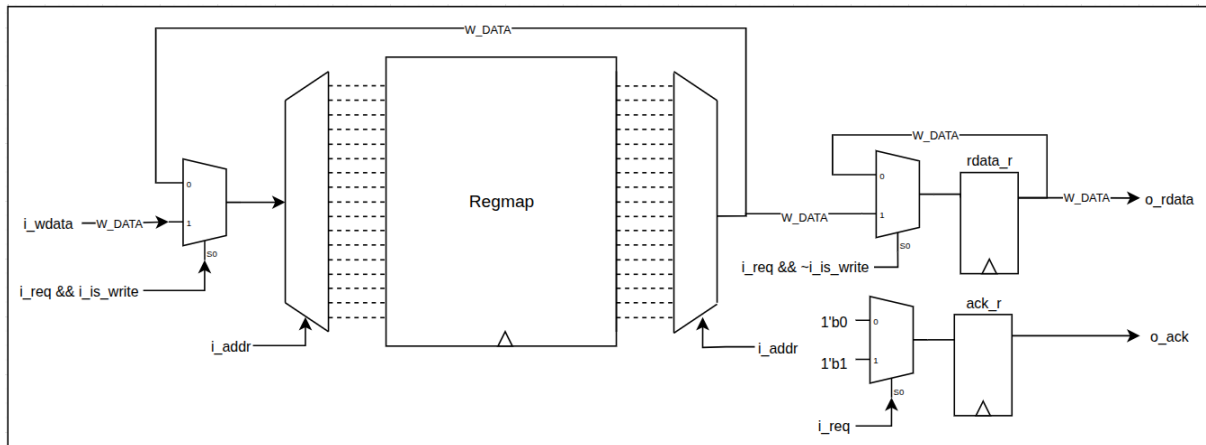


Figura 9: Arquitectura RTL simplificada del mapa de registros.

6.3. Transacciones de acceso

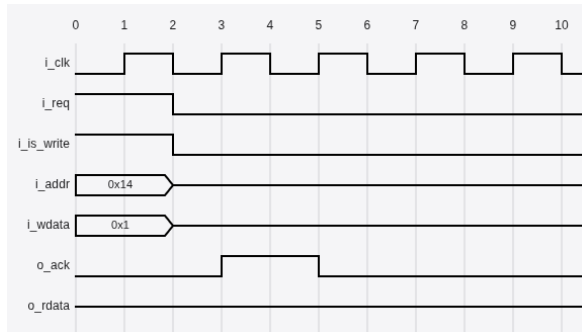
Escritura

Para realizar una escritura, el SW coloca la dirección y el dato sobre `i_addr` e `i_wdata`, respectivamente, y activa simultáneamente `i_req` e `i_is_write`. En el próximo flanco positivo del clock, el **regmap** decodifica la dirección y actualiza el registro correspondiente. La aceptación de la operación se informa mediante un pulso en `o_ack`, un ciclo de clock después del request.

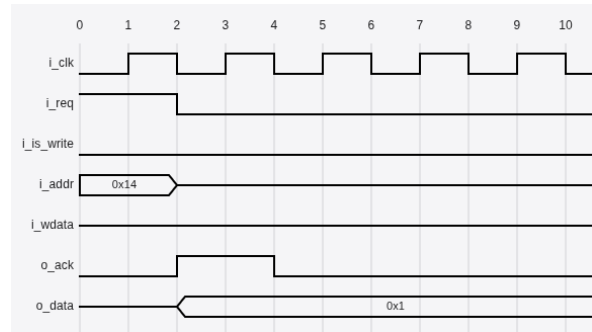
Lectura

En una lectura, el SW coloca la dirección y activa `i_req`, manteniendo `i_is_write` en bajo. El bloque decodifica la dirección, y al próximo flanco positivo del clock escribe el valor seleccionado en `o_rdata` y activa `o_ack` para indicar que el dato es válido. Las direcciones no implementadas devuelven cero.

La Figura 10 muestra las formas de onda de ambas operaciones. En los dos casos, la dirección y las señales de control se presentan antes del flanco de captura, mientras que la confirmación se observa un ciclo después.



(a) Secuencia de escritura.



(b) Secuencia de lectura.

Figura 10: Transacciones de acceso al mapa de registros.

La lectura de DU_RDDATA constituye un caso particular, debido a la latencia de la memoria de la DU. Cuando las cuatro unidades finalizaron la captura, el acceso genera el pedido de lectura y mantiene la transacción pendiente hasta recibir `i_du_rd_valid`. Recién entonces se entrega el dato y se activa `o_ack`. Si las DUs todavía no finalizaron, la lectura devuelve cero sin iniciar el acceso a memoria.

6.4. Mapa de direcciones

La Tabla 3 resume los registros accesibles por software y su interacción con el hardware.

Cuadro 3: Mapa de registros del sistema.

Addr	Registro	SW	HW	Descripción
0x00	PRBS_CTRL	R/W	R	bit[0] = START, inicio de secuencia bit[1] = STOP, detención de secuencia bit[4:2] = SEL, selección de PRBS10-15
0x01	PRBS_SEED	R/W	R	bit[14:0] = SEED inicial del LFSR
0x02	PRBS_STATUS	R	W	bit[0] = RUNNING, secuencia activa
0x10	FIR_TAP_0	R/W	R	bit[11:0] = coeficiente FIR h_0 , S(12,10)
0x11	FIR_TAP_1	R/W	R	bit[11:0] = coeficiente FIR h_1 , S(12,10)
0x12	FIR_TAP_2	R/W	R	bit[11:0] = coeficiente FIR h_2 , S(12,10)
0x13	FIR_TAP_3	R/W	R	bit[11:0] = coeficiente FIR h_3 , S(12,10)
0x14	FIR_TAP_4	R/W	R	bit[11:0] = coeficiente FIR h_4 , S(12,10)
0x15	FIR_TAP_5	R/W	R	bit[11:0] = coeficiente FIR h_5 , S(12,10)
0x16	FIR_TAP_6	R/W	R	bit[11:0] = coeficiente FIR h_6 , S(12,10)
0x17	FIR_TAP_7	R/W	R	bit[11:0] = coeficiente FIR h_7 , S(12,10)
0x18	FIR_TAP_8	R/W	R	bit[11:0] = coeficiente FIR h_8 , S(12,10)
0x19	FIR_TAP_9	R/W	R	bit[11:0] = coeficiente FIR h_9 , S(12,10)
0x1A	FIR_CTRL	R/W	R	bit[0] = COMMIT, transferir los coeficientes al filtro bit[1] = ENABLE del filtro
0x1B	FIR_STATUS	R	W	bit[0] = BUSY Vale uno mientras se encuentra en curso la transferencia de coeficientes. Los coeficientes deben permanecer estables durante este intervalo.
0x20	DU_CTRL	R/W	R	bit[0] = ARM, inicia o reinicia la adquisición bit[2:1] = MODE: PRE, MID y POST
0x21	DU_THRESHOLD	R/W	R	bit[11:0] = umbral de disparo, S(12,10)
0x22	DU_STATUS	R	W	bit[3:0] = DONE. bit 0 = DU0, bit 1 = DU1, bit 2 = DU2 y bit 3 = DU3
0x23	DU_RDADDR	R/W	R	bit[7:0] = dirección de lectura de la DU seleccionada

Continúa en la página siguiente.

Addr	Registro	SW	HW	Descripción
0x24	DU_RDDATA	R	W	bit[11:0] = muestra leída de la DU seleccionada0, S(12,10)
0x25	DU_RDPTR	R	W	bit[7:0] = posición de memoria inicial de lectura de la DU seleccionada
0x26	DU_SELECT	R/W	R	bit[1:0] = selector de la DU utilizada para lectura. 00 = DU0, 01 = DU1, 10 = DU2 y 11 = DU3.
0x30	CLKMEAS_CTRL	W	R	bit[0] = START, inicia nueva medición
0x31	CLKMEAS_WINDOW	R/W	R	bit[18:0] = largo de ventana de medición en cuentas del clock de referencia CPU
0x32	CLKMEAS_COUNT	R	W	bit[19:0] = cuentas del clock DSP durante la ventana de referencia
0x33	CLKMEAS_STATUS	R	W	bit[0] = DONE

6.5. Verificación

La verificación se realizó mediante una prueba de configuración y lectura utilizando el set de registros del frecuencímetro. En primer lugar, se forzó desde el testbench un resultado de cuenta y su señal de finalización, y se comprobó su lectura a través de CLKMEAS_COUNT y CLKMEAS_STATUS. Luego se escribió un valor de ventana en CLKMEAS_WINDOW, verificando tanto su propagación hacia la salida de configuración como su posterior lectura desde el bus.

En todos los accesos se comprobó la generación de o_ack, el valor de o_rdata en las lecturas y la correcta actualización de las salidas del frecuencímetro. De esta forma se validaron las operaciones básicas de escritura, lectura y consulta de estado del mapa de registros.

7. Integración del sistema (top)

7.1. Arquitectura e integración

El módulo **top** instancia los bloques presentados en las secciones anteriores y hace sus conexiones, con el especial cuidado de respetar los CDC. Como se observa en la Figura 11, el **regmap** recibe el bus externo en el dominio de CPU y distribuye la configuración hacia cada bloque, en su respectivo dominio de clock.

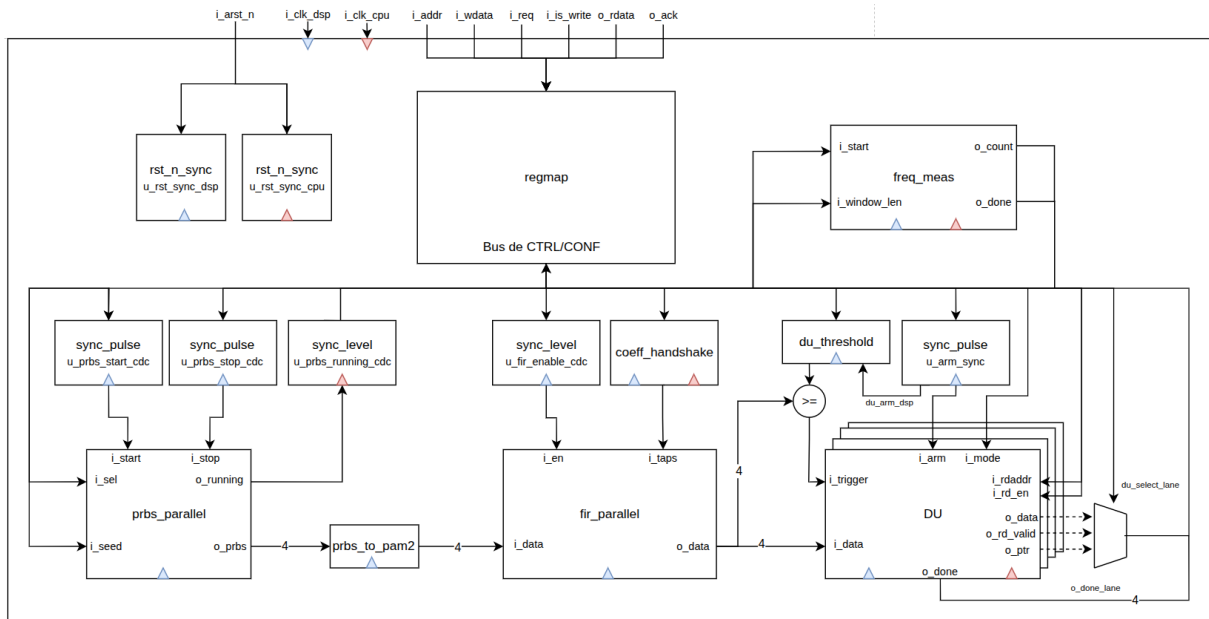


Figura 11: Interconexión de los bloques del sistema en el módulo **top**.

Reset Global

El sistema recibe un único reset asíncrono activo en bajo, `i_arst_n`. Su aserción se propaga de forma asíncrona a todo el diseño, mientras que su liberación se sincroniza independientemente en los dominios CPU y DSP mediante dos instancias de `rst_n_sync`. De esta manera, cada bloque abandona el estado de reset sobre un flanco válido de su propio clock.

Cruces de dominio de clock

Los pulsos `START` y `STOP` de la PRBS, y `ARM` de las DUs, cruzan desde CPU hacia DSP mediante sincronizadores de FF con detector de flanco, ya que al ir de un clock lento a uno rápido se debe transferir el pulso a la duración del clock rápido para evitar que dure mas de un ciclo de clock. Por otro lado, las señales de nivel `PRBS_RUNNING` y `FIR_ENABLE` utilizan sincronizadores de nivel con FF. La semilla y el selector de polinomio permanecen estables alrededor del pulso `START`, por lo que se transfieren como `bundled-data` y se capturan en el dominio DSP al iniciar la PRBS. De la misma manera, el modo y el umbral de las DUs se configuran antes de `ARM`, y se registran en el dominio DSP cuando llega dicho pulso.

El frecuencímetro resuelve internamente sus cruces de dominio. Pero se destaca que el resultado de cuenta se transfiere como `bundled-data` también, utilizando `o_done` como indicación de que el dato ya se encuentra estable.

Transferencia de coeficientes del FIR

Para cargar los coeficientes al filtro FIR, primero se deberán escribir los diez coeficientes independientemente en el regmap, donde se guardarán en los llamados registros *shadow*. Una escritura del bit `COMMIT` activará la transacción utilizando el bloque `coeff_handshake` y poniendo el bit `BUSY` en alto. Durante este periodo, no se podrá modificar el valor de los coeficientes en el regmap, ya que se estarán registrando en el dominio de DSP. Una vez que se termina la transacción, se libera `BUSY` y ahora los coeficientes cargados en los registros llamados *active* serán los utilizados por el filtro FIR.

Este mecanismo implementa un cruce de `bundled-data`: mientras la transferencia está en curso, los registros *shadow* permanecen estables y el handshake garantiza que el filtro actualice todos los coeficientes en el mismo evento, sin mezclar valores pertenecientes a configuraciones diferentes.

Datapath y captura

El bloque `prbs_to_pam2` convierte cada bit de la salida paralela de la PRBS en una muestra con signo de dos bits. El bit uno se representa mediante `+1`, mientras que el bit cero se representa mediante `-1`. Las cuatro muestras resultantes ingresan simultáneamente al FIR paralelo.

Por otro lado, las cuatro salidas del FIR se comparan con un valor umbral común, que fue previamente registrado en el dominio de DSP al momento del armado de la DU. Si al menos una de ellas resulta mayor o igual que el umbral, se genera un único trigger compartido para las cuatro DUs:

$$\text{du_trigger_dsp} = \bigvee_{i=0}^3 (\text{fir_odata_dsp}[i] \geq \text{du_threshold_dsp}). \quad (25)$$

Esto permite que las cuatro lanes comiencen su captura al mismo tiempo. Para la lectura de cada DU, `du_select_lane` habilita las salidas correspondientes a cada lane: `o_ptr`, `o_data` y `o_rd_valid`. Se optó hacerlo de esta forma para reducir tamaño de buses ya que podrían generar una gran congestión. En cambio, las cuatro señales `o_done_lane` se conservan por separado para informar el estado completo de la captura.

7.2. Verificación

La verificación del sistema integrado se realizó mediante un entorno UVM basado en clases, cuya organización se muestra en la Figura 12. La secuencia genera las transacciones de configuración y lectura; el driver las convierte en accesos al bus del `regmap`; el monitor observa pasivamente las operaciones completadas; y el scoreboard comprueba que los estados y resultados obtenidos sean coherentes con la secuencia ejecutada. La comunicación entre estos componentes se implementó mediante `mailboxes` e interfaces virtuales.

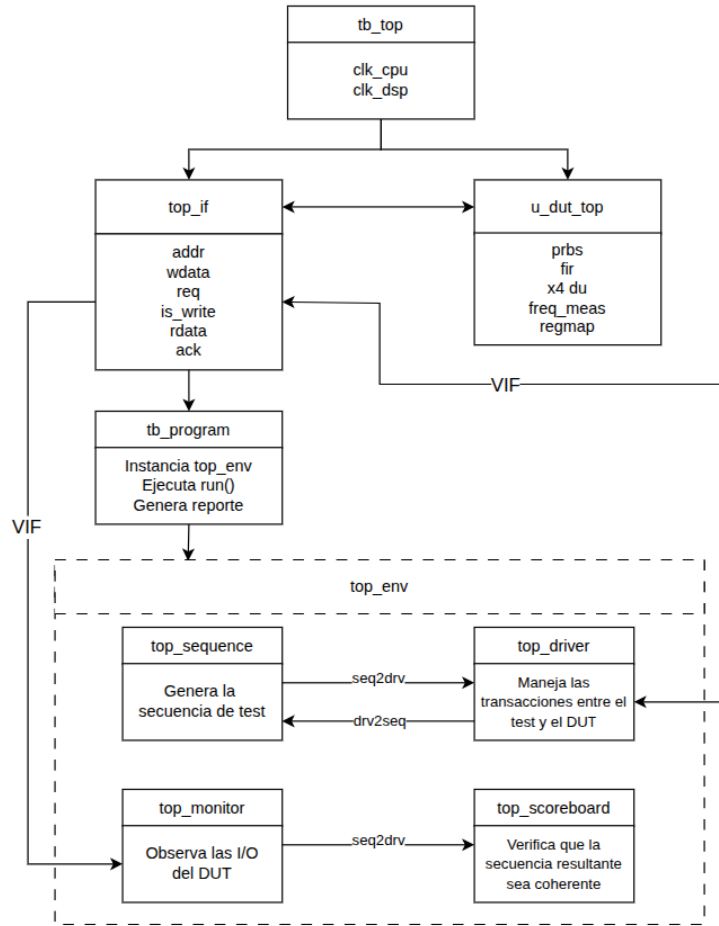


Figura 12: Entorno de verificación del sistema integrado.

La secuencia del test comienza aplicando el reset y configurando todos los bloques desde el mapa de registros. Se carga una semilla 0xFFFF y se selecciona PRBS15; se escriben los diez coeficientes independientes del FIR y se ejecuta su COMMIT, verificando la secuencia BUSY-READY. También se configura la DU en modo POST con umbral cero y se programa la ventana del frecuencímetro. Antes de iniciar el procesamiento, se leen nuevamente los registros para comprobar su configuración.

En primer lugar, se hace la medición del clock de 1 GHz con el de 200 MHz, y se comprueba que el resultado esté dentro del rango deseado. Luego se inicia la PRBS, se verifica su estado RUNNING en alto, se habilita el FIR y se arman las cuatro DUs. Una vez que las cuatro indican DONE, se selecciona cada lane por separado y se leen su puntero y las 256 posiciones de memoria. El scoreboard comprueba que se hayan escrito la 4 DU por completo y que no contengan solo ceros. Para terminar, se detiene la PRBS, se verifica la desactivación de RUNNING y se deshabilita el filtro.

7.3. Resultados de síntesis

La síntesis se evaluó a partir de cuatro métricas: timing, área, potencia y cantidad de tipos de celdas segun tensión de umbral. Estos resultados permiten comprobar el cumplimiento de frecuencia y reconocer qué bloques y tipos de celda dominan el costo del diseño.

Timing

El diseño cumple las restricciones de timing para ambos clocks, de 200 MHz y 1 GHz.

El critical path pertenece al dominio del clock más rápido, de 1 GHz, y queda identificado por los siguientes puntos:

- **Startpoint:** u_fir_parallel/o_data_reg.
- **Endpoint:** clock_gate_gen_du_lane.

Siendo su WNS:

Cuadro 4: WNS del top		
Métrica	Valor	Unidad
Required time	0.95	ns
Arrival time	-0.91	ns
Slack	0.04 (MET)	ns

Por lo tanto, el camino crítico comienza en los registros de salida del filtro FIR y finaliza en la lógica de clock gating, probablemente asociado a **w_en** del modulo DU.

Área

La Figura 13 presenta la distribución del área entre los principales bloques. El área total obtenida fue de $21\,060\,\mu\text{m}^2$. Las cuatro DUs ocupan en conjunto $18\,600\,\mu\text{m}^2$, equivalentes al 88 % del total, debido principalmente a las memorias necesarias para almacenar las muestras. El FIR queda en segundo lugar, con $1\,600\,\mu\text{m}^2$, mientras que el resto proviene del PRBS, el frecuencímetro, regmap y otros.

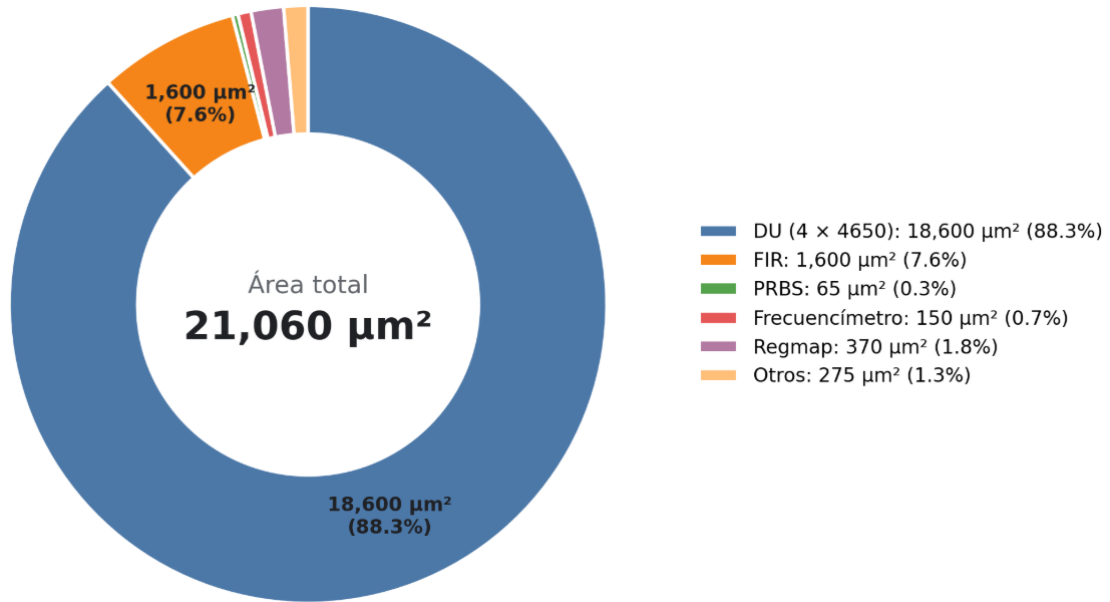


Figura 13: Distribución del área sintetizada entre los bloques del sistema.

Potencia

La Tabla 5 resume el reporte de potencia total, entre los principales bloques.

Cuadro 5: Potencia estimada para el sistema integrado.

Bloque	Potencia	Proporción
DU	1.2mW	68.5 %
Others	0.55mW	31.5 %
Total	1.75mW	100 %

Distribución de celdas por tensión de umbral

La proporción de celdas según su tensión de umbral permite analizar el compromiso realizado por la herramienta entre velocidad y potencia. Las celdas de menor tensión de umbral ofrecen mayor velocidad

a costa de un mayor leakage, mientras que las de mayor tensión de umbral reducen el leakage pero son más lentas.

Cuadro 6: Distribución de celdas según su tensión de umbral.

Tipo de celda	Cantidad	Proporción
HVT	26009	98.7 %
RVT	15	0.06 %
LVT	304	1.15 %
SLVT	21	0.08 %
Total	26349	100 %

8. Conclusiones

En este trabajo se diseñaron y verificaron los distintos bloques que componen el sistema, para luego integrarlos en un único módulo **top**. La arquitectura obtenida permite generar una secuencia PRBS paralela, convertirla a PAM-2, procesarla mediante un filtro FIR y capturar sus cuatro salidas utilizando las DUs. A su vez, el mapa de registros permite configurar y controlar el sistema desde el dominio CPU, mientras que el frecuencímetro permite comprobar la frecuencia del clock DSP.

Los resultados obtenidos fueron satisfactorios. Las simulaciones presentaron un comportamiento acorde con el funcionamiento diseñado y esperado, tanto para cada bloque como para el sistema completo. Los resultados de síntesis también fueron satisfactorios: el diseño cumple las restricciones de timing para los clocks de 1 GHz y 200 MHz, con valores de área y potencia razonables para los distintos bloques del sistema. El área se encuentra dominada por las cuatro DUs debido a sus memorias, mientras que la potencia total estimada es de 1,75 mW. Por otro lado, resulta llamativa la elevada proporción de celdas HVT, pudiendo indicar que existe margen para cerrar timing a frecuencias mayores, aunque evaluar ese límite no forma parte del objetivo de este trabajo.

La principal dificultad encontrada consistió en dejar de considerar cada bloque como una caja negra y comenzar a pensarlo como parte de un sistema completo. Esto requirió tener en cuenta en qué momento y de qué manera se configura cada módulo, además de la relación entre sus señales de control y de datos. La existencia de dominios de clock diferentes para la configuración y el datapath agregó complejidad a estas transacciones.

Como posible mejora, sería interesante evaluar una tecnología que permita inferir memorias en lugar de implementar el almacenamiento mediante grandes bancos de registros. Esto permitiría comprobar si es posible reducir el área y la potencia de las DUs, que constituyen el bloque dominante en ambas métricas.

El balance final del trabajo es muy positivo, ya que permitió comprender mejor el funcionamiento de un ASIC desde una perspectiva global y tomar decisiones de diseño desde las primeras etapas, considerando su impacto sobre el funcionamiento del sistema como un todo.